

Reviewer Pack — Minimal Rank of Quantized Matrix Entropy vs SAT Satisfiability...

Entry 5502db6438d8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 19:12:36 UTC

Minimal Rank of Quantized Matrix Entropy vs SAT Satisfiability

Entry ID: 5502db6438d8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 19:12:36 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum Entropy) **Field B** (complexity object): Complexity Theory: SAT Satisfiability

Statement:

[‘For every satisfiable 3-CNF formula F with m clauses and n variables, define the quantum entropy $H(F)$ as the von Neumann entropy of its associated density matrix. Then, the minimal rank of the matrix representing the quantum entropy of F is upper-bounded by a polynomial function in n and m .’, ‘For all satisfiable 3-CNF formulas F with $n \leq 40$ variables, there exists a constant c such that the rank of the matrix representing $H(F)$ is at most $cn + m$.’]

Rationale (proposer’s reasoning):

[‘Quantum entanglement is known to be related to computational complexity. The use of quantum entropy as an invariant could potentially expose hidden structures in SAT formulas.’, ‘The quantum information theory perspective on classical computation might lead to novel insights into the nature of NP-complete problems and their solutions.’]

Taxonomy category: Quantum_Entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9bf24ce0b62d2598

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of the quantum entropy matrix for satisfiable 3-CNF formulas with $n \leq 40$ variables should be bounded by $cn + m$, where c is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 10 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - quantum entropy AND von Neumann entropy AND upper-bound polynomial n m - 3-CNF formula AND satisfiability AND minimal rank quantum information theory - polynomial bound on rank matrix entanglement complexity theory SAT

Top relevant hits considered: - [http://arxiv.org/abs/2311.09985v2] Measurement of forward charged hadron flow harmonics in peripheral PbPb collisions at $\sqrt{s_{NN}} = 5.02$ TeV with the - [http://arxiv.org/abs/math-ph/0102013v1] Entropy, von Neumann and the von Neumann entropy - [http://arxiv.org/abs/1809.06500v3] Range entropy: A bridge between signal complexity and self-similarity - [http://arxiv.org/abs/2402.03554v3] Explicit Formula for Partial Information Decomposition - [http://arxiv.org/abs/1810.06456v2] The Parthasarathy formula and a spectral triple for the quantum Lagrangian Grassmannian of rank two - [http://arxiv.org/abs/1512.04514v2] Information Structures for Feedback Capacity of Channels with Memory and Transmission Cost: Stochastic Optimal Control & - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/1307.2323v2] Bounding Polynomial Entanglement Measures for Mixed States

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, m):
        clauses = []
        for _ in range(m):
            clause = set()
            while len(clause) < 3:
                var = random.randint(1, n)
                if var not in clause and -var not in clause:
                    clause.add(var)
            clauses.append(tuple(sorted(clause)))
        return clauses

    def density_matrix(clauses):
        n = max(abs(v) for v in set().union(*clauses))
        dm = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in clauses:
            for i in range(3):
```

```

        for j in range(i + 1, 3):
            x, y = clause[i], clause[j]
            dm[abs(x)][abs(y)] += 1
            dm[abs(y)][abs(x)] += 1
    return dm

def von_neumann_entropy(dm):
    n = len(dm)
    trace = sum(dm[i][i] for i in range(n))
    if trace == 0:
        return 0
    dm = [[dm[i][j] / trace for j in range(n)] for i in range(n)]
    eigenvalues = [0] * n
    for _ in range(100): # Power iteration method
        v = [random.random() for _ in range(n)]
        v = [v_i / sum(v) for v_i in v]
        v_next = [sum(dm[i][j] * v[j] for j in range(n)) for i in range(n)]
        v_next = [v_next_i / sum(v_next) for v_next_i in v_next]
        eigenvalues[0] += abs(sum(v_i * v_next_i for v_i, v_next_i in zip(v, v_next)))
    return -sum(eigenvalue * math.log2(eigenvalue) if eigenvalue > 0 else 0 for eigenvalue in

def matrix_rank(dm):
    m = len(dm)
    n = len(dm[0])
    dm_copy = [row[:] for row in dm]
    rank = 0
    for i in range(min(m, n)):
        if dm_copy[i][i] != 0:
            for j in range(i + 1, m):
                factor = dm_copy[j][i] / dm_copy[i][i]
                for k in range(n):
                    dm_copy[j][k] -= factor * dm_copy[i][k]
            rank += 1
        else:
            found_pivot = False
            for j in range(i + 1, m):
                if dm_copy[j][i] != 0:
                    dm_copy[i], dm_copy[j] = dm_copy[j], dm_copy[i]
                    found_pivot = True
                    break
            if not found_pivot:
                continue
            for j in range(i + 1, m):
                factor = dm_copy[j][i] / dm_copy[i][i]
                for k in range(n):
                    dm_copy[j][k] -= factor * dm_copy[i][k]
            rank += 1
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    m = random.randint(1, n * (n - 1) // 2)
    formula = generate_3cnf(n, m)
    dm = density_matrix(formula)
    entropy = von_neumann_entropy(dm)

```

```

rank = matrix_rank(dm)
results.append({
    "n": n,
    "m": m,
    "entropy": entropy,
    "rank": rank
})

metric_name = "rank_bound"
metric_value = sum(result["rank"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(result["rank"] <= 2 * result["n"] + result["m"] for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

name: 'rank_bound', 'metric_value': 20.0, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''
TRIAL: {'metric_name': 'rank_bound', 'metric_value': 18.0, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_bound', 'metric_value': 20.0, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_bound', 'metric_value': 17.833333333333332, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_bound', 'metric_value': 19.666666666666668, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_bound', 'metric_value': 19.666666666666668, 'instances_tested': 6, 'conjecture_holds': True, 'counterexample': ''}

```

TRIAL: {'metric_name': 'rank_bound', 'metric_value': 20.0, 'instances_tested': 6, 'conjecture_holds':
 TRIAL: {'metric_name': 'rank_bound', 'metric_value': 19.5, 'instances_tested': 6, 'conjecture_holds':
 TRIAL: {'metric_name': 'rank_bound', 'metric_value': 19.666666666666668, 'instances_tested': 6, 'con
 TRIAL: {'metric_name': 'rank_bound', 'metric_value': 20.0, 'instances_tested': 6, 'conjecture_holds':
 TRIAL: {'metric_name': 'rank_bound', 'metric_value': 19.833333333333332, 'instances_tested': 6, 'con
 TRIAL: {'metric_name': 'rank_bound', 'metric_value': 17.666666666666668, 'instances_tested': 6, 'con
 TRIAL: {'metric_name': 'rank_bound', 'metric_value': 18.833333333333332, 'instances_tested': 6, 'con
 TRIAL: {'metric_name': 'rank_bound', 'metric_value': 19.833333333333332, 'instances_tested': 6, 'con
 RESULT: SUPPORTED mean=19.472222222222222 std=0.680527210294061 support_fraction=1.0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The empirical test only includes 6 instances, which is too small to be conclusive. The metric may not scale trivially with n and m , and the conjecture might fail for larger instances.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test only includes 6 instances, which is too small to be conclusive according to the critic's reasoning that the metric may not scale trivially with n and m . | next: Increase the number of tested instances to at least 100 to provide a more robust conclusion.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11486
2	preregistration	ollama_remote	glm4:latest	0	0	5272
3	novelty	ollama_remote	glm4:latest	0	0	4868
4	novelty	ollama_remote	glm4:latest	0	0	5656
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17491
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11025
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10420
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30786
9	critic	ollama_remote	glm4:latest	0	0	49052
10	judge	ollama_remote	glm4:latest	0	0	6022

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 152079 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5502db6438d8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/5502db6438d8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5502db6438d8.tar.gz` (if generated)